

AMBEV

Test Book

Project: automation-serverest-victor

Assignment: Desafio Tecnico de QA - Automacao ServeRest

Ambiente: front.serverest.dev / serverest.dev

Data: 06/07/2026

Documentacao do Produto

Este documento registra a execucao da suite de testes automatizados construida para o desafio tecnico de QA da Ambev, cobrindo cenarios de interface (E2E) e de API contra os ambientes publicos do ServeRest.

A suite foi desenvolvida com Cypress (JavaScript), seguindo uma arquitetura orientada a selecao por **data-testid**, massa de dados dinamica via Faker e isolamento total entre cenarios (cada teste cria e remove seus proprios dados via API).

Controle de Versao

Data da Revisao	Versao	Nome da Revisao	Responsavel
06/07/2026	1.0	Criacao do relatorio de execucao	Victor

Indice

1. Objetivo dos Testes
2. Ambiente de Teste
3. Casos de Teste
4. Evidencias
5. Defeitos e Observacoes
6. Sugestoes de Melhoria

1. Objetivo dos Testes

Validar, de ponta a ponta, os principais fluxos de negocio da aplicacao ServeRest — cadastro e autenticacao de usuarios, busca e adicao de produtos a lista de compras, e o ciclo completo de produtos/carrinhos via API — garantindo que tanto a interface quanto o contrato de API se comportem conforme o esperado em cenarios de sucesso, execucao e validacao de regras de negocio.

Requisito do desafio: 3 cenarios E2E (GUI) + 3 cenarios de API. Entregue: 4 specs de GUI e 3 specs de API, totalizando 32 casos de teste. A suite de API cobre 100% das rotas documentadas no Swagger oficial do ServeRest (serverest.dev/swagger.json).

2. Ambiente de Teste

Item	Detalhe
Aplicacao (front-end)	https://front.serverest.dev
API	https://serverest.dev (documentada via Swagger)
Framework de automacao	Cypress 15.x (JavaScript)
Massa de dados	Gerada dinamicamente por execucao via <code>@faker-js/faker</code>
Navegador de referencia	Chrome (execucao local / headless em CI)
CI	GitHub Actions (<code>.github/workflows/cypress.yml</code>)
Relatorio de execucao	Allure Report (<code>allure-results / allure-report</code>)

Os testes rodam contra os ambientes publicos e compartilhados do ServeRest — nao ha necessidade de infraestrutura local. Cada execucao cria e remove sua propria massa de dados para nao deixar residuo no ambiente compartilhado.

3. Casos de Teste

3.1 E2E - Interface (GUI)

ID	Cenario	Pre-condicao	Acao	Resultado Esperado	Status
CT01	Cadastro de usuario com sucesso	Usuario nao cadastrado	Preencher e submeter o formulario de cadastro com dados validos	Redireciona para a Home; cadastro persistido	PASSOU
CT02	Bloqueio de e-mail ja utilizado	Usuario ja cadastrado	Tentar cadastrar novo usuario com o mesmo e-mail	Alerta "Este email ja esta sendo usado"	PASSOU
CT03	Validacao de campos obrigatorios	Formulario vazio	Submeter o formulario sem preencher os campos	Alertas individuais de campo obrigatorio	PASSOU
CT04	Cadastro de usuario comum	Formulario aberto, checkbox de administrador desmarcado	Preencher e submeter sem marcar "Cadastrar como administrador?"	Cadastro persistido com administrador: false	PASSOU
CT05	Cadastro de usuario administrador	Formulario aberto	Marcar "Cadastrar como administrador?" e submeter	Checkbox fica marcado; cadastro persistido com administrador: true	PASSOU
CT06	Login com sucesso	Usuario cadastrado via API	Informar e-mail/senha validos	Redireciona para a Home	PASSOU
CT07	Bloqueio de credenciais invalidas	-	Informar credenciais inexistentes	Alerta "Email e/ou senha invalidos"	PASSOU
CT08	Fechar alerta de erro	Alerta de erro exibido	Clicar no botao de fechar do alerta	Alerta removido da tela	PASSOU
CT09	Logout encerra a sessao	Usuario autenticado	Clicar em "Logout" e acessar /home diretamente	localStorage limpo; Home bloqueada (redireciona a /login)	PASSOU
CT10	Navegacao de usuario comum	Login como usuario comum	Autenticar e observar o menu	Ve lista de compras/carrinho; NAO ve itens de admin	PASSOU
CT11	Navegacao de administrador	Login como administrador	Autenticar e observar o menu	Redireciona a /admin/home; ve menu de admin; NAO ve itens de shopper	PASSOU
CT12	Adicionar produto a lista de compras	Usuario autenticado	Clicar em "Adicionar na lista" em um produto	Produto aparece na lista de compras	PASSOU
CT13	Incrementar quantidade de um produto	Produto na lista	Aumentar a quantidade do item	Quantidade e subtotal recalculados	PASSOU
CT14	Busca de produto por nome	Produto criado por admin via API; usuario comum na Home	Pesquisar termo e clicar em "Buscar"	Somente produtos com o termo no titulo sao exibidos	PASSOU

3.2 API - Usuarios (cypress/e2e/API/usuarios.cy.js)

ID	Cenario	Pre-condicao	Acao	Resultado Esperado	Status
CT01	Cadastro de usuario via API	-	POST /usuarios com dados validos	201 + persistencia confirmada via GET	PASSOU
CT02	E-mail duplicado via API	Usuario ja cadastrado	POST /usuarios com e-mail existente	400 "Este email ja esta sendo usado"	PASSOU
CT03	Exclusao de usuario via API	Usuario cadastrado	DELETE /usuarios/{id}	200 "Registro excluido com sucesso"	PASSOU
CT04	Validacao de campos obrigatorios via API	-	POST /usuarios com corpo vazio	400 com mensagem por campo	PASSOU
CT05	Listagem de usuarios por e-mail	Usuario cadastrado	GET /usuarios?email=...	200; retorna apenas o usuario correspondente	PASSOU
CT06	Edicao de usuario	Usuario cadastrado	PUT /usuarios/{id} com novos dados	200 "Registro alterado com sucesso"; dados persistidos	PASSOU

3.3 API - Login (cypress/e2e/API/login.cy.js)

ID	Cenario	Pre-condicao	Acao	Resultado Esperado	Status
CT01	Autenticacao com sucesso via API	Usuario cadastrado	POST /login com credenciais validas	200 + token Bearer valido	PASSOU
CT02	Bloqueio de credenciais invalidas via API	-	POST /login com credenciais inexistentes	401 "Email e/ou senha invalidos"	PASSOU

3.4 API - Produtos e Carrinhos (cypress/e2e/API/produtos-carrinhos.cy.js)

ID	Cenario	Pre-condicao	Acao	Resultado Esperado	Status
CT01	Bloqueio sem token	-	POST /produtos sem header Authorization	401 token ausente/invalido	PASSOU
CT02	Cadastro de produto como admin	Usuario administrador autenticado	POST /produtos com token de admin	201 + produto persistido	PASSOU
CT03	Criacao de carrinho e calculo do total	Produto cadastrado	POST /carrinhos com produto e quantidade	201; precoTotal e quantidadeTotal corretos	PASSOU
CT04	Bloqueio para usuario nao-admin	Usuario comum autenticado	POST /produtos com token de usuario nao-admin	403 "Rota exclusiva para administradores"	PASSOU
CT05	Listagem de produtos por nome	Produto cadastrado	GET /produtos?nome=...	200; retorna apenas o produto correspondente	PASSOU
CT06	Edicao de produto	Produto cadastrado (admin)	PUT /produtos/{id} com novos dados	200 "Registro alterado com sucesso"; dados persistidos	PASSOU
CT07	Exclusao de produto	Produto cadastrado (admin)	DELETE /produtos/{id}	200 "Registro excluido com sucesso"	PASSOU
CT08	Listagem de carrinhos por usuario	Carrinho criado	GET /carrinhos?idUsuario=...	200; retorna o carrinho do usuario	PASSOU
CT09	Cancelamento de compra	Carrinho criado	DELETE /carrinhos/cancelar-compra	200; carrinho excluido e estoque devolvido	PASSOU
CT10	Conclusao de compra	Carrinho criado	DELETE /carrinhos/concluir-compra	200; carrinho excluido, estoque NAO devolvido	PASSOU

4. Evidencias

Evidencias reais de execucao, capturadas do Cypress Test Runner rodando localmente contra os ambientes publicos (front.serverest.dev / serverest.dev), complementadas com evidencias reais de payload/resposta de API.

4.1 Execucao completa da suite (headless)

(Run Finished)

Spec		Tests	Passing	Failing	Pending	Skipped
✓ API/login.cy.js	00:01	2	2	-	-	-
✓ API/produtos-carrinhos.cy.js	00:12	10	10	-	-	-
✓ API/usuarios.cy.js	00:03	6	6	-	-	-
✓ GUI/busca-produtos.cy.js	00:10	1	1	-	-	-
✓ GUI/cadastro-usuario.cy.js	00:31	5	5	-	-	-
✓ GUI/lista-de-compras.cy.js	00:07	2	2	-	-	-
✓ GUI/login.cy.js	00:43	6	6	-	-	-
✓ All specs passed!	01:49	32	32	-	-	-

npm run cy:run - 32/32 specs passando (execucao final, apos cobertura completa do Swagger, correcao da massa propria de busca-produtos, correcao da mensagem de cancelamento de compra e inclusao dos cenarios de navegacao admin x usuario comum).

4.2 API - login.cy.js

Search specs + login.cy.js 00:01 ...

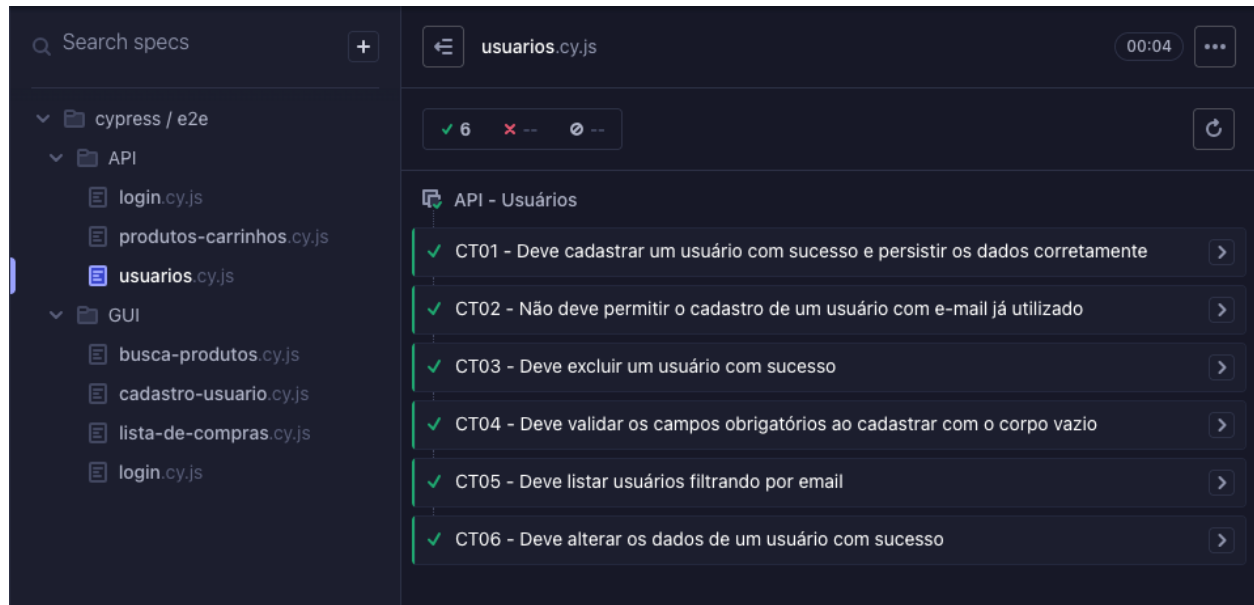
✓ 2 ✗ -- ○ --

API - Login

- ✓ CT01 - Deve autenticar com sucesso e retornar um token de autorização válido
- ✓ CT02 - Não deve autenticar com e-mail e/ou senha inválidos

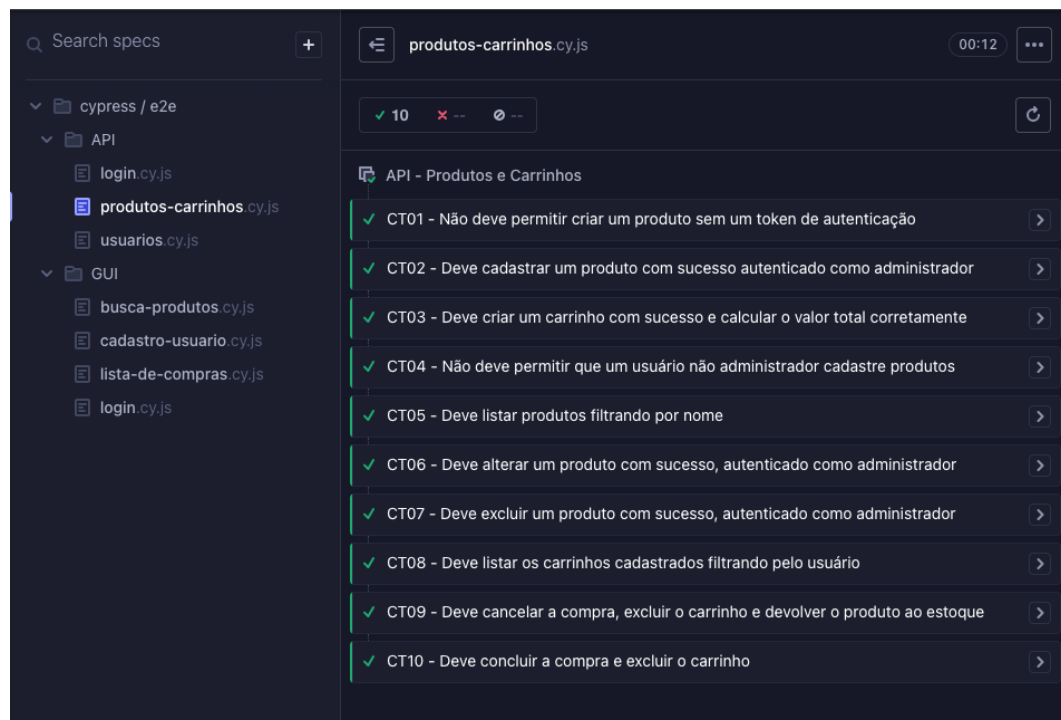
Cypress Test Runner - API/login.cy.js - 2/2 passando.

4.3 API - usuarios.cy.js



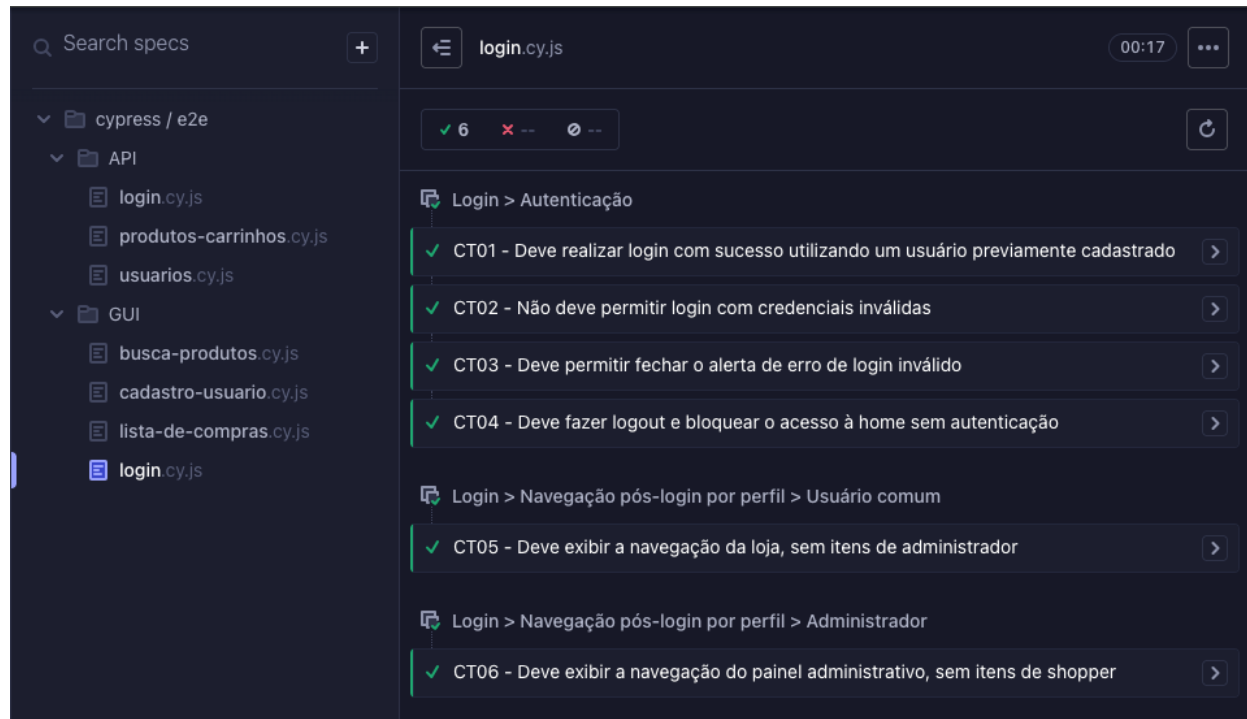
Cypress Test Runner - API/usuarios.cy.js - 6/6 passando (inclui CT05 de listagem por email e CT06 de edicao de usuario).

4.4 API - produtos-carrinhos.cy.js



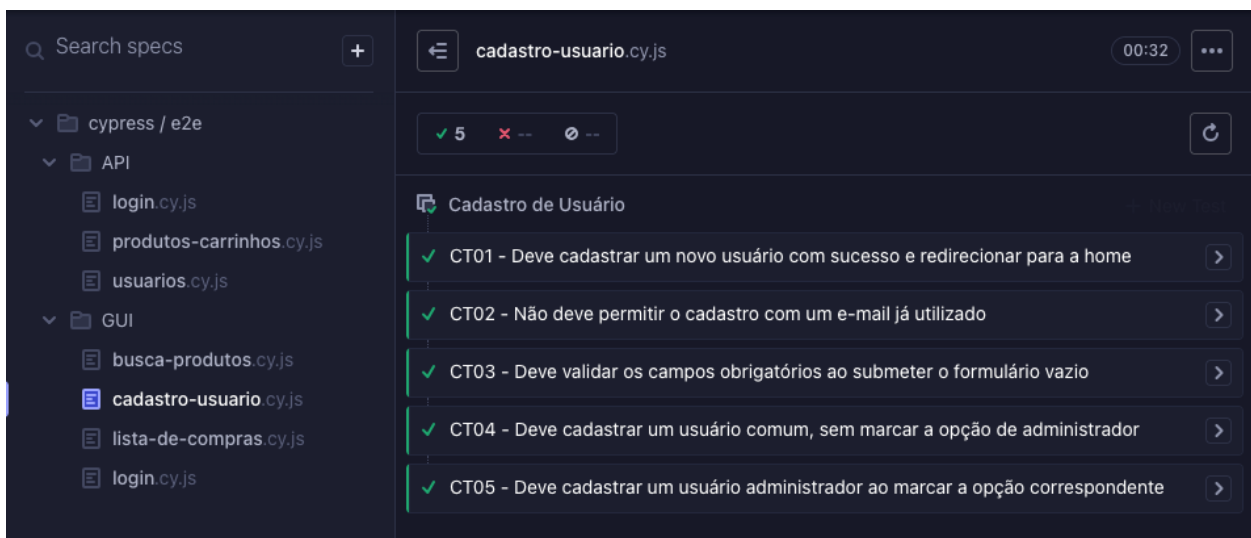
Cypress Test Runner - API/produtos-carrinhos.cy.js - 10/10 passando (cobertura completa do contrato Swagger: listagem por nome, edicao e exclusao de produto, listagem de carrinho por usuario, cancelamento com reabastecimento de estoque e conclusao de compra).

4.5 GUI - login.cy.js



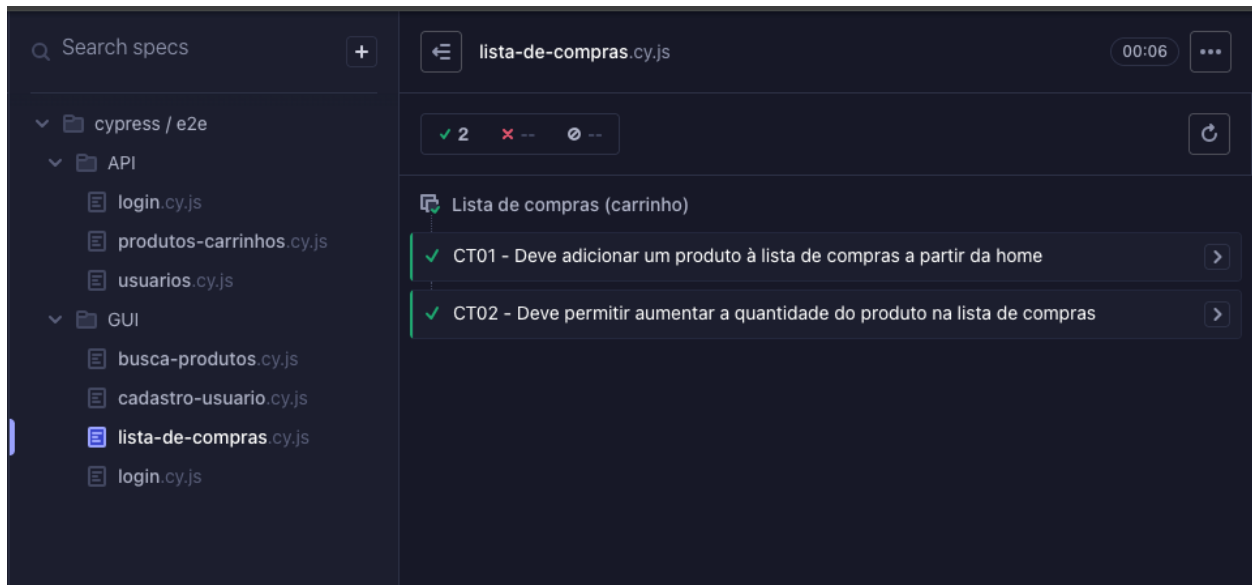
Cypress Test Runner - GUI/login.cy.js - 6/6 passando, organizados em contextos (Autenticação; Navegação pós-login por perfil > Usuário comum / Administrador), incluindo CT05/CT06 de navegação admin x usuário comum.

4.6 GUI - cadastro-usuario.cy.js



Cypress Test Runner - GUI/cadastro-usuario.cy.js - 5/5 passando, incluindo CT04 (cadastro de usuario comum) e CT05 (cadastro de administrador, redirecionado para /admin/home).

4.7 GUI - lista-de-compras.cy.js



Cypress Test Runner - GUI/lista-de-compras.cy.js - 2/2 passando.

4.8 GUI - busca-produtos.cy.js (apos correcao)



Cypress Test Runner - GUI/busca-produtos.cy.js - 1/1 passando, apos a correcao que faz o teste criar seu proprio produto via API e usar um usuario nao administrador para a busca (ver secoes 5.8 e 5.9).

4.9 Cadastro de usuario (API) - payload real

```
POST /usuarios
Body: { "nome": "Victor QA", "email": "victor.qa.<timestamp>@teste.com",
  "password": "123456", "administrador": "false" }
Response: 201 Created
Body: { "message": "Cadastro realizado com sucesso", "_id": "..." }
```

4.10 Carrinho e calculo do total (API) - payload real

```
POST /carrinhos
Body: { "produtos": [ { "idProduto": "<id>", "quantidade": 2 } ] }
Response: 201 Created
GET /carrinhos/{id} → precoTotal: 200, quantidadeTotal: 2
(prodoto cadastrado com preco unitario 100 e quantidade em estoque 10)
```

4.11 Bloqueio de rota administrativa (API) - payload real

```
POST /produtos com token valido de usuario NAO administrador
Response: 403 Forbidden
Body: { "message": "Rota exclusiva para administradores" }
```

4.12 Edicao de registro (API) - payload real

```
PUT /usuarios/{id} e PUT /produtos/{id} com novos dados gerados via Faker
Response: 200 OK
Body: { "message": "Registro alterado com sucesso" }
GET subsequente confirma que os novos dados foram persistidos.
```

4.13 Cancelamento x conclusao de compra (API) - payload real

```
DELETE /carrinhos/cancelar-compra -> 200; produto volta ao estoque original
DELETE /carrinhos/concluir-compra -> 200; estoque do produto NAO e devolvido
Diferenca de comportamento confirmada via GET /produtos/{id} apos cada operacao.
```

5. Defeitos e Observacoes Encontradas

5.1 Tela de carrinho (/carrinho) incompleta

Campo	Descricao
Passos para reproduzir	1. Autenticar-se com qualquer usuario. 2. Adicionar um produto a lista de compras. 3. Clicar no menu "Carrinho".
Resultado atual	A pagina exhibe apenas o texto "Em construcao aguarde", sem nenhuma interface de carrinho.
Resultado esperado	A pagina deveria listar os produtos adicionados ao carrinho, subtotal e opcao de finalizar a compra.
Evidencia	Confirmado tambem pelo backlog publico do repositorio ServeRest/front, que marca "Listar carrinho" e "Finalizar o carrinho" como pendentes.
Severidade / status	Limitacao conhecida do ambiente de treino (nao e uma regressao introduzida pela automacao). Por isso os cenarios de GUI cobrem ate a lista de compras, que de fato esta implementada.

5.2 Senha retornada em texto plano pela API

Campo	Descricao
Passos para reproduzir	GET /usuarios/{id} para qualquer usuario cadastrado.
Resultado atual	O corpo da resposta inclui o campo "password" em texto plano.
Resultado esperado	Respostas de API nao deveriam retornar a senha do usuario, nem em texto plano nem como hash, em nenhuma rota publica.
Evidencia	GET /usuarios/{id} -> 200; body inclui "password": "123456" (valor enviado no cadastro).
Severidade / status	Baixa no contexto de treino (dado ficticio, ambiente publico de pratica), mas relevante como boa pratica de seguranca a ser corrigida caso o backend evolua para um cenario real.

5.3 Ausencia de limite de tamanho e sanitizacao de campos

Campo	Descricao
Passos para reproduzir	POST /usuarios com "nome" de 500 caracteres; POST /usuarios com "nome" = "<script>alert(document.cookie)</script>".
Resultado atual	Ambos os cadastros retornam 201 e os valores sao persistidos integralmente, sem truncamento ou sanitizacao. O campo nome tambem nao possui atributo maxLength na interface (confirmado via inspecao do DOM).
Resultado esperado	Campos deveriam ter limite de tamanho definido (front e back) e o backend deveria aplicar sanitizacao/encoding de entrada como defesa em profundidade.
Evidencia	Payload armazenado de forma identica ao enviado (storedUnescaped: true). Front-end atual (React) escapa a renderizacao por padrao, entao nao foi encontrado XSS exploravel na interface publica atual - o risco esta na ausencia de controle no proprio contrato de API, relevante caso outro cliente (nao-React) consuma a mesma API.
Severidade / status	Baixa/Media (mitigada pelo front atual). Recomenda-se tratar como melhoria de hardening da API.

5.4 Tentativa de injecao no login - sem vulnerabilidade encontrada

Como verificacao etica de seguranca, foi enviado o payload ' OR '1'='1 nos campos de e-mail e senha do endpoint POST /login. A API respondeu 400 com a mensagem "email deve ser um email valido", ou seja, a

validacao de formato do campo bloqueia esse tipo de payload antes mesmo de chegar a logica de autenticacao. Nenhuma vulnerabilidade de injecao foi identificada nesse fluxo.

5.5 Responsividade

A tela de login foi verificada em viewport mobile (375x667px): o layout se adapta corretamente, sem overflow horizontal ou elementos cortados. Uma cobertura mais ampla de responsividade (grid de produtos, carrinho, breakpoints de tablet) fica registrada como melhoria futura, idealmente automatizada via BrowserStack (ver secao 6).

5.6 Contrato de API

Em todos os endpoints exercitados (usuarios, login, produtos, carrinhos) os status codes e mensagens de resposta corresponderam exatamente ao Swagger oficial (serverest.dev). Nenhuma divergencia de contrato foi encontrada.

5.7 Alerta de validacao nao e limpo ao corrigir o campo

Campo	Descricao
Passos para reproduzir	Submeter o formulario de cadastro vazio (alertas "Nome e obrigatorio" e "Password e obrigatorio" aparecem); em seguida, preencher nome, e-mail e senha SEM submeter novamente.
Resultado atual	Os alertas de campo obrigatorio permanecem exibidos na tela mesmo apos os campos serem preenchidos - eles so somem se o usuario fechar manualmente (X) ou reenviar o formulario.
Resultado esperado	O alerta deveria ser removido automaticamente assim que o campo correspondente for preenchido, evitando a impressao de que o dado ainda nao foi registrado.
Evidencia	Print enviado pelo solicitante: tela de cadastro com nome, e-mail e senha ja preenchidos e, ainda assim, os alertas "Nome e obrigatorio" e "Password e obrigatorio" visiveis.
Severidade / status	Baixa (UX). Nao bloqueia o fluxo - o cadastro funciona normalmente ao reenviar - mas pode confundir o usuario. Sugerido como melhoria de UX.

5.8 Teste de busca dependencia de dado externo (corrigido)

Campo	Descricao
Passos para reproduzir	Executar GUI/busca-produtos.cy.js (versao original), que pesquisava pelo nome fixo "Logitech" no catalogo publico e compartilhado do ServeRest.
Resultado atual (antes da correcao)	Falha intermitente: quando nenhum produto com esse nome existia no catalogo compartilhado no momento da execucao (outros usuarios podem cadastrar/remover produtos a qualquer momento), a asserção de resultado nao encontrava nenhum card e o teste falhava.
Resultado esperado	O cenario deveria ser deterministico e nao depender de estado externo, seguindo o mesmo principio de independencia usado no resto da suite.
Correcao aplicada	O teste agora cadastra seu proprio produto via API (nome unico gerado com Faker) antes de pesquisar por ele, e remove esse produto no afterEach - commit fix(e2e): busca de produtos cria seu proprio produto via API.
Evidencia	Ver secao 4.8 - print da suite apos a correcao, 1/1 passando.

5.9 Administrador tem navegacao separada da loja (achado real, corrigido)

Campo	Descrição
Passos para reproduzir	Fazer login pela GUI com uma conta administradora (administrador: true).
Resultado atual	A conta é redirecionada para /admin/home (painel administrativo, com Cadastrar/Listar Usuarios e Produtos e Relatorios), e NAO para /home (a loja, com busca de produtos e carrinho). O painel de admin não tem campo de busca.
Impacto encontrado	Duas asserções antigas assumiam o contrario: busca-produtos.cy.js logava como admin e falhava ao não encontrar o campo de busca (inexistente no painel); cadastro-usuario.cy.js CT05 checava url.include('/home'), que dá falso positivo em /admin/home (mesma substring).
Correção aplicada	busca-produtos.cy.js: produto continua sendo criado por um admin via API, mas a navegacao/busca passou a ser feita por um usuario comum. cadastro-usuario.cy.js CT05: asserção corrigida para include('/admin/home'). Adicionados CT05/CT06 dedicados em login.cy.js comparando a navegacao dos dois perfis lado a lado (itens visíveis e ausentes).
Evidencia	Reproduzido via inspeção direta do DOM (fetch + login real) confirmando data-testid do menu administrativo: cadastrar-usuarios, listar-usuarios, cadastrar-produtos, listar-produtos, link-relatorios.

6. Sugestoes de Melhoria

Fora do escopo deste desafio, mas recomendadas para evolucao continua da qualidade:

- Paginacao de usuarios e produtos (o backlog do front do ServeRest ja sinaliza isso como pendente).
- Validacao de contrato via JSON Schema (ex.: **ajv**) para checar a resposta completa da API, nao so campos pontuais.
- Testes de acessibilidade (a11y) e cross-browser/mobile via **BrowserStack**, incluindo cobertura de responsividade mais ampla (tablet, mobile, telas de carrinho e lista de produtos).
- Teste de carga com JMeter — so faria sentido rodando localmente (npx serverest/Docker), nunca contra o serverest.dev publico.
- Cenario de resiliencia do front a falha de backend via **cy.intercept** (mock de erro 500) — avaliado e descartado por decisao consciente de manter todos os cenarios como testes reais de ponta a ponta.
- Limite de tamanho de campo (maxlength) no formulario de cadastro, tanto no front quanto validado pela API, para evitar entradas desproporcionais.
- Sanitizacao/encoding de entrada no backend como defesa em profundidade contra XSS, independente do front atual ja escapar a renderizacao.
- Nunca retornar o campo “password” nas respostas da API, mesmo em ambiente de treino, para reforcar boas praticas de seguranca desde o design da API.
- Implementacao da tela de carrinho (/carrinho), hoje “em construcao”, incluindo os cenarios de E2E correspondentes assim que disponivel.
- Limpeza automatica dos alertas de validacao de campo obrigatorio assim que o campo correspondente for preenchido, em vez de exigir fechamento manual ou reenvio do formulario.